

Module 23.12: Summary of Linear Regression Foundations

1. Unified Mathematical Synthesis

Linear Regression is the foundational supervised learning algorithm for continuous estimation. The entire framework rests upon three unified pillars:

1. Hypothesis (Model Structure):

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = \sum_{j=1}^d w_j x_j + b$$

2. Objective Function (Cost Landscape):

$$J(\mathbf{w}, b) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} + b\mathbf{1} - \mathbf{y}\|_2^2$$

3. Optimization Solver: Finding $\mathbf{w}^* = \arg \min J(\mathbf{w}, b)$.

2. Analytical Solver vs. Iterative Solvers

There are two primary paradigms to find the optimal parameter vector $\mathbf{w}^*$:

```
+-----+
| Paradigm 1: Analytical Closed-Form (Normal Equation)
| Formula:  $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ 
| Complexity:  $O(d^3)$  matrix inversion
+-----+
```

vs.

```
+-----+
| Paradigm 2: Numerical Optimization (Gradient Descent)
| Formula:  $\mathbf{w} := \mathbf{w} - \alpha * (1/m) \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$ 
| Complexity:  $O(k * m * d)$  where  $k$  is iterations
+-----+
```

Comprehensive Comparison Matrix

Attribute	Analytical Normal Equation	Gradient Descent (Numerical)
Hyperparameters	None (α or iterations not needed)	Requires tuning α and iteration count
Feature Scaling	Not required	Strongly recommended (Standardization)
Scalability in Features (d)	Slow when $d > 10,000$ due to $\mathcal{O}(d^3)$ inversion	Scales gracefully to $d > 10^7$ features
Scalability in Samples (m)	Memory intensive for massive datasets	Scales via mini-batch and streaming SGD
Algorithmic Nature	One-shot exact algebraic solution	Iterative numerical descent

3. The 3 Variants of Gradient Descent

1. Batch Gradient Descent (BGD)

Computes gradient over ALL m training instances per step.

* Path: Deterministic, smooth descent.

* Drawback: Computationally expensive per step when m is large.

2. Stochastic Gradient Descent (SGD)

Computes gradient over ONE randomly sampled instance per step.

* Path: Highly erratic, zig-zagging trajectory.

* Advantage: Instantaneous updates, can bounce out of shallow local dips.

3. Mini-Batch Gradient Descent (MBGD)

Computes gradient over a small subset B (e.g., 32, 64, 128 instances).

* Path: Balanced, moderately smooth trajectory.

* Advantage: Leverages vectorized GPU hardware; industry standard.

4. The Critical Role of Feature Scaling

When features have radically disparate scales (e.g., $x_1 \in [0, 1]$ vs $x_2 \in [1000, 500000]$):

- The cost function contours stretch into extremely **elongated, narrow ellipses**.
- Gradient vectors point almost perpendicularly to the path of the true minimum, causing severe oscillations and slow convergence.
- Standardizing features ($\mu = 0, \sigma = 1$) reshapes the contours into symmetric **concentric circles**, allowing gradient descent to march directly toward the minimum along the shortest path.